OBJEKTNO-ORIJENTISANO PROGRAMIRANJE · SMJEROVI B, C, D

Životni vijek objekta, kopiranje i operatori

objekat na heapu · destruktor · plitka i duboka kopija · vektor objekata · prijatelji klase

1.1 · OBJEKAT NA HEAP-U

Dva su mjesta na kojima objekat može nastati – poznato iz jezika C

Automatsko trajanje — stek

Racun a("501-001", "Marko", 1000);

Objekat nastaje na mjestu deklaracije i prestaje da postoji kada kontrola izvršavanja izađe iz bloka. O tome se brine prevodilac, programer nema kontrolu nad životnim vijekom objekta

Dinamičko trajanje — heap

Racun* p = new Racun("501-002", "Ana", 500); //u jeziku C imali smo malloc

Objekat nastaje kada se naiđe na *new* i traje dok ga ne obrišemo sa *delete*. Oba trenutka bira programer i odgovoran je za brisanje objekta (oslobađanje memorije)

Zašto su nam potrebni objekti na heap-u

Kada broj objekata nije poznat pri pisanju programa, nego zavisi od onoga što se dogodi tokom izvršavanja. Npr. Banka ne zna koliko će računa biti otvoreno danas, pa ih ne može deklarisati unaprijed.

Operatori new i delete

```
Racun::Racun(std::string broj, std::string ime, double pocetno)
    : broj(broj), vlasnik(ime), stanje(pocetno > 0 ? pocetno : 0)
{
    brojRacuna++;
    std::cout << "  [konstruktor " << broj << "]\n";
}


Racun::~Racun()
{
    brojRacuna--;
    std::cout << "  [destruktor " << broj << "]\n";
}


int main()
{
    std::cout << "a nastaje na steku:\n";

    Racun a("501-001", "Marko Markovic", 1000);

    std::cout << "p pokazuje na objekat na heapu:\n";

    Racun* p = new Racun("501-002", "Ana Anic", 500);

    p->uplata(250);
    p->prikazi();

    std::cout << "Ukupan broj racuna: " << Racun::getBrojRacuna() << "\n";

    std::cout << "delete p:\n";

    delete p;

    std::cout << "Ukupan broj racuna: " << Racun::getBrojRacuna() << "\n";
    std::cout << "kraj funkcije main:\n";
}
```

ispis programa

```
a nastaje na steku:
  [konstruktor 501-001]
p pokazuje na objekat na heapu:
  [konstruktor 501-002]
501-002  Ana Anic: 750 EUR
Ukupan broj racuna: 2
delete p:
  [destruktor 501-002]
Ukupan broj racuna: 1
kraj funkcije main:
  [destruktor 501-001]
```

poziv destruktora objekta a, na samom kraju izvršavanja funkcije main

new zauzme memoriju za objekat, pozove konstruktor i vrati adresu objekta
new nije isto što i **malloc** - malloc samo zauzima bajtove, ne poziva konstruktor

delete pozove destruktor i oslobodi memoriju

Neke greške koje prevodilac ne prijavljuje

Zaboravljen delete

Objekat ostaje u memoriji do kraja programa, a nijedan pokazivač više ne ukazuje na njega. To je **curenje memorije (memory leak)**.

Dvostruki delete

Druga naredba **delete** nad istom adresom izaziva pucanje programa.

Upotreba poslije brisanja

Pokazivač poslije **delete** i dalje sadrži staru adresu, ali objekta tamo nema. Poziv metode preko tog pokazivača ponekad radi a ponekad ne – nepredvidivo ponašanje

Pravilo

Svaki **new** ima tačno jedan **delete**, na mjestu koje je unaprijed poznato. Ako se ne može reći gdje će objekat biti obrisan, ne treba ga ni praviti sa **new**.

Primjer za zaboravljen delete

Zaboravljen delete

Objekat ostaje u memoriji do kraja programa, a nijedan pokazivač više ne ukazuje na njega. To je **curenje memorije (memory leak)**.

```
int main()
{
    Racun* p = new Racun("501-001", „Marko Marković", 1000);

    p->uplata(250);

    p = new Racun("501-002", "Ana Anic", 500); // ovdje smo trajno izgubili pristup prvom objektu (Marko)

    p->prikazi();

    delete p;
}
```

Destruktor - metoda koja se poziva na kraju životnog vijeka objekta

```
class Racun {
    // ...
public:
    Racun(std::string broj, std::string ime,
          double pocetno = 0.0);
    ~Racun();              // destruktor
};

Racun::~Racun()
{
    brojRacuna--;
    std::cout << "  [destruktor " << broj << "]\n";
}
```

Nosi ime klase sa znakom **~ (tilda)** ispred.
Nema povratni tip, čak ni **void**.
Nema argumente, pa ga nije moguće preklopiti.
Klasa ima **najviše jedan** destruktor.

Destruktor je jedina metoda koju nikad ne pozivamo eksplicitno.
Poziva je samo i uvijek prevodilac:

- na kraju bloka (za objekte na steku) ili
- pri pozivu operatora delete (za objekte na heap-u)

Trenutak poziva destruktora i obrnuti redoslijed

```
int main()
{
        Racun a("501-001", "Marko Markovic", 1000);
        Racun b("501-002", "Ana Anic", 500);
        {
            Racun c("501-003", "Pero Peric", 300);
            std::cout << "izlazimo iz unutrasnjeg bloka:\n";
        }
        std::cout << „Ukupni broj racuna: "
                  << Racun::getBrojRacuna() << "\n";
}
```

ispis programa

```
  [konstruktor 501-001]
  [konstruktor 501-002]
  [konstruktor 501-003]
izlazimo iz unutrasnjeg bloka:
  [destruktor 501-003]
Ukupni broj racuna: 2
kraj main:
  [destruktor 501-002]
  [destruktor 501-001]
```

Objekat **c** prestaje da postoji na zatvorenoj vitičastoj zagradi, a ne na kraju funkcije.

Objekti **a** i **b** nestaju u **obrnutom** redoslijedu od nastanka. Objekat koji je posljednji nastao prvi nestaje.

Unaprijedili smo funkcionalnost statičkog atributa brojRacuna

Ranije u našem kodu statički atribut **brojRacuna** samo je rastao u konstruktoru. Svaki poziv konstruktora uvećavao je brojač ali nismo ga smanjivali kada objekat nestane.

```
class Racun {
    // ...
public:
    Racun(std::string broj, std::string ime,
          double pocetno = 0.0);
    ~Racun();              // destruktor
};

// definicija destruktora
Racun::~Racun()
{
    brojRacuna--;
    std::cout << "  [destruktor " << broj << "]\n";
}
```

Sa destruktorom ćemo to unaprijediti

Destruktor se poziva pri **nestajanju** računa dakle u trenutku umanjivanja ukupnog broja računa, pa brojač treba u tim trenucima umanjivati.

U destruktor pišemo sve što treba uraditi kada objekat prestaje da postoji.

Pravilo: memoriju koju konstruktor zauzme, destruktor treba da oslobodi

```
class Banka {
    std::string ime;
    Racun* glavniRacun; // ukazivaće objekat koji će klasa Banka sama napraviti

public:
    Banka(std::string ime)
        : ime(ime),
          glavniRacun(new Racun("501-000", "Glavni racun", 0))
    {
    }

    ~Banka()
    {
        // klasa koja je napravila glavniRacun, ona ga i brise.
        // bez delete imali bismo curenje memorije (zauzeta memorija
        // koja nikada nije oslobođena)

        delete glavniRacun;
    }

    void prikazi() const { glavniRacun->prikazi(); }
};

int main()
{
    Banka b("Prva banka");
    b.prikazi();
    std::cout << "kraj main:\n";
}
```

ispis programa

```
[konstruktor 501-000] 501-000
Glavni racun: 0 EUR
kraj main:
[destruktor 501-000]
```

Korisnik klase Banka (u ovom slučaju funkcija main) ne mora znati da negdje u metodama klase Banka postoji **kreiranje objekta klase Racun**. Banka pravi objekat klase Racun i koristi ga i ona je odgovorna za njegovo oslobadjanje sa delete. U C++ se taj obrazac zove upravljanje resursom kroz objekat.

Situacije u kojima se poziva specijalna vrsta konstruktora: konstruktor kopije

Ima i drugih načina da nastane objekat osim deklaracijom ili pozivom operatora new, kao što je bio slučaj u dosadašnjim primjerima. Npr.:

Inicijalizacija objekta

Izvod b = a;
Izvod b(a);

Oba zapisa prave novi objekat (b) od postojećeg objekta (a).

Prenos argumenta po vrijednosti

void f(Izvod i)

Argument je lokalni objekat funkcije: nastaje kopiranjem onoga što je predato pri pozivu funkcije.

Vraćanje objekta iz funkcije

Izvod f() { ... return x; }

Nastaje privremeni objekat koji prevodilac često ukloni, pa se ovdje konstruktor kopije ne mora vidjeti u ispisu.

Privremeni objekti

Bezimeni objekti koje prevodilac pravi i uništava sam, bez našeg znanja. Standard jezika mu dozvoljava da ih izbjegne, pa se broj poziva konstruktora kopije može razlikovati od prevodioca do prevodioca. Zato se u konstruktor kopije nikada ne stavlja kod koji mora da se izvrši tačno jednom.

Inicijalizacija jednog objekta drugim objektom

```
Racun a("501-001", "Marko Markovic", 1000);

Racun b = a;        // ugradjeno kopiranje, clan po clan
b.uplata(250);

a.prikazi();
b.prikazi();
```

ispis programa

```
  [konstruktor 501-001]
501-001  Marko Markovic: 1000 EUR // poziv metode prikazi()
501-001  Marko Markovic: 1250 EUR // poziv metode prikazi()
  [destruktor 501-001] // na kraju main-a poziva se destruktor
  [destruktor 501-001] // na kraju main-a poziva se destruktor
```

U ispisu imamo **jedan** red od konstruktora i **dva** od destruktora. Objekat **b** je nastao, a nijedan naš konstruktor nije pozvan zato što je prevodilac sam napravio i pozvao tzv. **konstruktor kopije** koji prepisuje atribut po atribut iz objekta a u objekat b. Takav konstruktor kopije koga pravi prevodilac zove se **ugrađeni konstruktor kopije.**

Za klasu Racun ovakvo prosto kopiranje atributa je u redu - dobili smo 2 računa a i b, svaki sa svojom kopijom broja računa, imena i stanja.

Međutim, problem nastaje kada je makar jedan od atributa klase pokazivač.

Klasa koja ima atribut tipa pokazivača

Izvod čuva niz iznosa. Tek pri kreiranju znamo koliko iznosa ima, pa niz mora biti na heapu.

Izvod.h

```
#ifndef IZVOD_H
#define IZVOD_H

class Izvod {
    // niz na heapu
    double* iznosi;
    int broj;

public:
    Izvod(int broj);
    ~Izvod();

    void postavi(int i, double x)
    {
        iznosi[i] = x;
    }
    double uzmi(int i) const
    {
        return iznosi[i];}
    };
#endif
```

Izvod.cpp

```
#include "Izvod.h"
#include <iostream>

Izvod::Izvod(int broj) : iznosi(new double[broj]),
broj(broj)
{
    std::cout << "[konstruktor, niz na adresi " <<
          iznosi << "]\n";
    for (auto i = 0; i < broj; i++) iznosi[i] = 0;
}
Izvod::~Izvod()
{
    std::cout << "  [destruktor, oslobadja " <<
    iznosi << "]\n";
    delete[] iznosi;
}
```

main.cpp

```
#include <iostream>
#include "Izvod.h"

int main()
{
 Izvod a(3);
 a.postavi(0, 120.5);

 std::cout << "a: " << a.uzmi(0) << "\n";
 std::cout << "kraj main:\n";
}
```

ispis programa:

```
[konstruktor, niz na adresi 0x55bbdba452b0]
a: 120.5
kraj main:
  [destruktor, oslobadja 0x55bbdba452b0]
```

new double[broj] zauzima niz i vraća pokazivač na taj niz
delete[] iznosi sa uglastim zagradama oslobađa niz *iznosi*

Za sad je sve u redu - konstruktor zauzima memoriju, destruktor oslobadja. Program sa jednim objektom tipa *Izvod* radi bez problema. Međutim problem nastaje kada napravimo **drugi objekat**, i to kopiranjem prvog

Klasa koja ima atribut tipa pokazivača

Izvod.h

```
#ifndef IZVOD_H
#define IZVOD_H

class Izvod {
    // niz na heapu
    double* iznosi;
    int broj;

public:
    Izvod(int broj);
    ~Izvod();

    void postavi(int i, double x)
    {
        iznosi[i] = x;
    }
    double uzmi(int i) const
    {
        return iznosi[i];}
    };
#endif
```

Izvod.cpp

```
#include "Izvod.h"
#include <iostream>

Izvod::Izvod(int broj) : iznosi(new double[broj]),
broj(broj)
{
    std::cout << "[konstruktor, niz na adresi " <<
          iznosi << "]\n";
    for (auto i = 0; i < broj; i++) iznosi[i] = 0;
}
Izvod::~Izvod()
{
    std::cout << "  [destruktor, oslobadja " <<
    iznosi << "]\n";
    delete[] iznosi;
}
```

Plitka kopija

Ugrađeni konstruktor kopije pravi **plitku kopiju**. Kopira se **pokazivač**, a ne ono na šta on pokazuje. Kopija je ostala na površini objekta i nije sišla do podataka na heapu.

Posljedica je da Izmjena nad **b** se vidi i na **a**, jer je niz jedan. Dodatno, na kraju main-a oba destruktora oslobadjaju **istu adresu**, pa program puca.

main.cpp

```
#include <iostream>
#include "Izvod.h"

int main()
{
 Izvod a(3);
 a.postavi(0, 120.5);

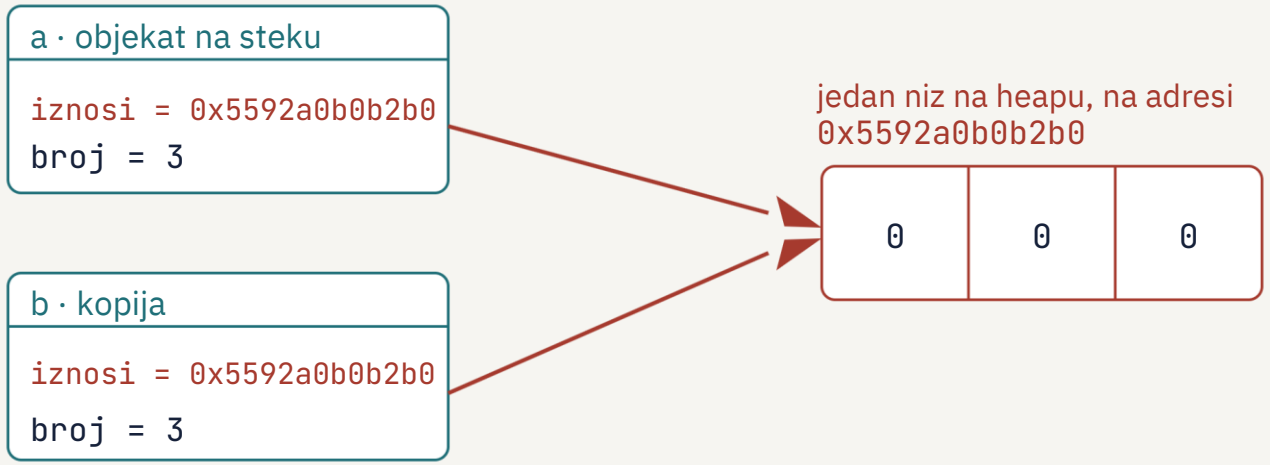
 Izvod b = a;  // ugradjeno kopiranje

//sljedeća linija mijenja i objekat a
 b.postavi(0, 999.0);
 std::cout << "a: " << a.uzmi(0) << "\n";
 std::cout << „b: " << b.uzmi(0) << "\n";
 std::cout << "kraj main:\n";
}
```

ispis programa:

```
[konstruktor, niz na adresi 0x5592a0b0b2b0]
a: 999
b: 999
kraj main:
  [destruktor, oslobadja 0x5592a0b0b2b0]
  [destruktor, oslobadja 0x5592a0b0b2b0]
free(): double free detected in tcache 2
```

Plitka kopija: 2 objekta i 2 pokazivača koji ukazuju na jedan isti niz



dva atributa su kopirana (*broj* i *iznosi*) pa oba objekta sada drže istu adresu na niz

Dva vlasnika jednog resursa

Niz ima dva vlasnika koji ne znaju jedan za drugoga. Svaki će ga, kada dođe vrijeme, osloboditi taj isti niz — a smije ga osloboditi samo jedan.

Da bi se ovaj problem prevazišao, praksa je da projektant klase piše konstruktor kopije, koji će se pozivati umjesto ugrađenog konstruktora kopije

Konstruktor kopije

Izvod.h

```
#ifndef IZVOD_H
#define IZVOD_H

class Izvod {
    // niz na heapu
    double* iznosi;
    int broj;

public:
    Izvod(int broj);
    // konstruktor kopije
    Izvod(const Izvod& drugi);
    ~Izvod();

    void postavi(int i, double x
    {
        iznosi[i] = x;
    }
    double uzmi(int i) const
    {
        return iznosi[i];}
    };
#endif
```

Izvod.cpp

```
#include "Izvod.h"
#include <iostream>

Izvod::Izvod(int broj) : iznosi(new double[broj]),
broj(broj)
{
    std::cout << "  [konstruktor, niz na " <<
    iznosi << "]\n";
    for (auto i = 0; i < broj; i++) iznosi[i] = 0;
}
Izvod::Izvod(const Izvod& drugi)
    : iznosi(new double[drugi.broj]), broj(drugi.broj)
{
    for (int i = 0; i < broj; i++)
        iznosi[i] = drugi.iznosi[i];

    std::cout << "  [konstruktor kopije, novi niz na "
<< iznosi << "]\n";
}
Izvod::~Izvod()
{
    std::cout << "  [destruktor, oslobadja " <<
    iznosi << "]\n";
    delete[] iznosi;
}
```

main.cpp ostaje isti

```
#include <iostream>
#include "Izvod.h"

int main()
{
 Izvod a(3);
 a.postavi(0, 120.5);
// kopiranje konstruktorom kopije
 Izvod b = a;

//sljedeća linija NE mijenja i obj. a
 b.postavi(0, 999.0);
 std::cout << "a: " << a.uzmi(0) << "\n"
 std::cout << "a: " << a.uzmi(0) << "\n"
 std::cout << "kraj main:\n";
}
```

```
 [konstruktor, niz na 0x55e2db8cd2b0]
  [konstruktor kopije,
novi niz na 0x55e2db8cd2d0]
a: 120.5
b: 999
kraj main:
  [destruktor, oslobadja 0x55e2db8cd2d0]
  [destruktor, oslobadja 0x55e2db8cd2b0]
```

Konstruktor kopije

Konstruktor kopije prima **referencu na konstantan objekat iste klase**.

Referenca je obavezna. Kad bi primao objekat po vrijednosti, a ne po referenci, za taj argument bi se pravila kopija pri njegovoj predaji konstruktoru, pa bi se pozivao konstruktor kopije bez kraja.

Duboka kopija

Ne kopira se pokazivač, nego ono na šta on pokazuje.

Dva različita niza. Destruktori oslobadjaju **dvije različite** adrese pa nema pucanja programa .

Program koji koristi klasu se nije mijenjao

Funkcija **main** je ostala ista: promijenili smo klasu, ispravili problem, a kod koji je poziva je ostao isti.

Predavanje argumenta pomoću const Izvod&

Ako hoćemo da predamo argument po **vrijednosti**, da bismo izbjegli kopiranje, koristimo **const Racun&**. Referencom ne kopiramo objekat (predajemo njegovu adresu) a sa *const* osiguravamo da ga funkcija neće mijenjati. To je posebno korisno kod velikih objekata kod kojih ima puno podataka za kopiranje.

```
void poVrijednosti(Izvod kopija)
{
    kopija.prikazi();
}

void poKonstantnojReferenci(const Izvod& ref)
{
    ref.prikazi();
}

poVrijednosti(a);
poKonstantnojReferenci(a);
```

ispis programa

```
poziv po vrijednosti:
  [konstruktor kopije, novi niz na 0x...2f0]
501-001: 3 stavki, ukupno 460.5 EUR
  [destruktor, oslobadja 0x...2f0]
------------------------------------------------
poziv po konstantnoj referenci:
501-001: 3 stavki, ukupno 460.5 EUR
```

Prenos po vrijednosti

Prevodilac kreira novi objekat (lokalnu kopiju), poziva se konstruktor kopije koji kreira novi niz i kopira sve elemente u njega. Na kraju funkcije, lokalni objekat se uništava, pritom se poziva destruktor koji oslobadja tu memoriju. Za izvod od hiljadu stavki to je hiljadu prepisanih brojeva da bi se samo ispisali iznosi funkcijom *prikazi*.

Prenos po konstantnoj referenci

Vidimo u ispisu da nema ni jednog reda od konstruktora ni od destruktora. Funkcija radi nad originalom, a oznaka **const** je garancija da funkcija ne može da promijeni taj original iako mu ima pristup preko reference.

Ako klasa ima destruktor, vjerovatno joj treba definisati i konstruktor kopije i operator dodjele

Pravilo trojke (Rule of Three): Destruktor, konstruktor kopije i operator dodjele dolaze zajedno. Klasi kojoj je potreban jedan, gotovo uvijek su potrebna i druga dva.

Razlog za pravilo trojke

Sve 3 metode govore o tome šta se dešava sa resursom koji klasa drži. Destruktor ga **oslobadja**, konstruktor kopije ga **umnožava**, operator dodjele ga **modifikuje**. Ako se napiše samo jedna od ove 3 metode, za ostale dvije ostaju samo ugrađene verzije koje o resursu (npr. nizu) ne znaju ništa.

Operator dodjele

b = a; nije isto što i **Izvod b = a;**

b = a; // poziva se **operator dodjele**
Izvod b = a; // poziva se **konstruktor kopije**

Kod **b = a;** objekat b već postoji i već ima svoj niz, pa mora taj niz osloboditi prije nego što preuzme novi.
Kod **Izvod b = a;** niz se kreira po prvi put

Kako znamo trebaju li nam ove tri metode za našu klasu

Pogledati atribute. Ako je bilo koji od njih pokazivač na nešto što je **klasa sama zauzela**, treba imati sve tri pomenute metode. Ako nijedan atribut nije pokazivač, ne treba nijedna od ove tri metode — kao npr. što je bio slučaj sa klasom **Racun**.

Ista klasa bez ijednog pokazivača

```
class Izvod {
private:
    std::string racun;
    std::vector<double> iznosi;   // nemamo vise double*

public:
    Izvod(std::string racun, int broj)
        : racun(racun), iznosi(broj, 0.0) {}

    void postavi(int i, double x) { iznosi[i] = x; }

    double ukupno() const
    {
        double s = 0;
        for (double x : iznosi)
            s += x;
        return s;
    }
};
```

Nema destruktora, nema konstruktora kopije, nema **new** ni **delete u kodu**, nema atributa **broj**. Klasa radi ispravno i kopira se ispravno, a kraća je.

Ovo je moguće zato što **vector** iznutra radi upravo ono što smo mi napisali od nule: zauzima niz, kopira ga u konstruktoru kopije i oslobađa u destruktoru.

Elementi vektora mogu biti i korisničke klase, npr. Racun

Možemo držati onoliko računa koliko ih banka otvori,
bez ijedne alokacije sa **new** u našem kodu.

```
Racun::Racun(std::string broj, std::string ime, double pocetno)
    : broj(broj), vlasnik(ime), stanje(pocetno > 0 ? pocetno : 0)
{
    brojRacuna++;
    std::cout << "  [konstruktor " << broj << "]\n";
}


Racun::~Racun()
{
    brojRacuna--;
    std::cout << "  [destruktor " << broj << "]\n";
}
void Racun::uplata(double iznos)
{
    if (iznos > 0)
        stanje += iznos;
}
void Racun::prikazi() const
{
  std::cout << vlasnik << ": " << stanje << "EUR " << broj <<"\n";
}
```

```
int main()
{
    std::vector<Racun> racuni;
    racuni.reserve(2);

    racuni.push_back(Racun("501-001", "Marko M.", 1000));
    racuni.push_back(Racun("501-002", "Ana Anic", 500));

    for (Racun& r : racuni)
        r.uplata(100);

    for (const Racun& r : racuni)
        r.prikazi();

     std::cout << "u vektoru: " << racuni.size() <<
     ", postoji racuna: " << Racun::getBrojRacuna() << "\n";
}
```

Dvije petlje

Prva petlja mijenja objekte, pa uzima **Racun&**. Druga samo čita objekte, pa uzima **const Racun&**. Nema kopiranja ni kod jedne ni kod druge petlje. Bez reference bi svaki prolaz kopirao račun u promjenljivu r.

ispis programa

```
  [konstruktor 501-001]
  [destruktor 501-001]
  [konstruktor 501-002]
  [destruktor 501-002]
501-001  Marko M.: 1100 EUR
501-002  Ana Anic: 600 EUR
u vektoru: 2, postoji racuna: 0
```

Elementi vektora mogu biti i korisničke klase, npr. Racun

```
int main()
{
    std::vector<Racun> racuni;
    racuni.reserve(2);

    racuni.push_back(Racun("501-001", "Marko M.", 1000));
    racuni.push_back(Racun("501-002", "Ana Anic", 500));

    for (Racun& r : racuni)
        r.uplata(100);

    for (const Racun& r : racuni)
        r.prikazi();

     std::cout << "u vektoru: " << racuni.size() <<
     ", postoji racuna: " << Racun::getBrojRacuna() << "\n";
}
```

ispis programa

```
  [konstruktor 501-001]
  [destruktor 501-001]
  [konstruktor 501-002]
  [destruktor 501-002]
501-001  Marko M.: 1100 EUR
501-002  Ana Anic: 600 EUR
u vektoru: 2, postoji racuna: 0
```

racuni.reserve(2); je tu da vektor ne bi premještao elemente pri rastu, jer bi se tada pojavilo jos kopiranja pa bi ispis programa bio nerazumljiv.

Racun::getBrojRacuna() vraća 0 zato što pri pozivu push_back metode vektora, izvršava se sljedeći niz koraka

1. nastaje privremeni račun (npr. Marko M.) — ispis konstruktora, brojač raste na 1;
2. push_back dobija **referencu** na njega i ugrađenim konstruktorom kopije gradi novi objekat **u memoriji vektora** — nema ispisa i brojač se ne mijenja, jer ugrađeni konstruktor kopije samo prepisuje polja, ne zna za brojač i ne ispisuje nikakav tekst;
3. izraz se završava i privremeni objekat klase Racun se uništava — ispis destruktora i brojač pada na 0, a u vektoru ostaje da postoji račun

vector<Racun>

Dodajmo sljedeći konstruktor kopije:

```
Racun::Racun(const Racun& drugi)
    : broj(drugi.broj),
      vlasnik(drugi.vlasnik),
      stanje(drugi.stanje)
{
    std::cout << " [konstruktor kopije " <<
    broj << "]\n";

    brojRacuna++; // uvećanje brojača
}
```

ispis istog koda **poslije dodavanja konstruktora kopije**

```
  [konstruktor 501-001]
  [konstruktor kopije 501-001]
  [destruktor 501-001]
  [konstruktor 501-002]
  [konstruktor kopije 501-002]
  [destruktor 501-002]
501-001  Marko M.: 1100 EUR
501-002  Ana Anic: 600 EUR
u vektoru: 2, postoji racuna: 2
```

Sada se pri kreiranju objekata u memoriji vektora poziva konstruktor kopije koga smo dodali, umjesto ugrađenog.

Ovo je drugi mogući razlog da se konstruktor kopije napiše, pored onog sa pokazivačem: kada klasa održava **zajednički podatak**, ugrađeni konstruktor kopije ga ne održava pa bi trebalo napraviti konstruktor kopije.

Preklapanje običnih metoda

```
class Racun {
public:
    void prikazi() const;
    void prikazi(const std::string& naslov) const;
};

void Racun::prikazi() const
{
    std::cout << broj << "  " << vlasnik
              << ": " << stanje << " EUR\n";
}

void Racun::prikazi(const std::string& naslov) const
{
    std::cout << naslov << ": ";
    prikazi();              // poziva prvu verziju
}
```

Pravilo za preklopljene metode (metode istog imena)

Preklopljene metode razlikuju se po **broju** ili **tipu** argumenata. Razlika samo u povratnom tipu nije dovoljna. Odluka koja metoda će se pozvati se donosi pri prevođenju, prema argumentima poziva.

Dobra praksa

Druga verzija ne prepisuje ono što radi prva, nego je **poziva**. Tako u ovom primjeru ispis atributa postoji na jednom mjestu i mijenja se na jednom mjestu.

Ključna riječ friend

Primjer prijateljske funkcije koja je **globalna**

```
class Complex
{
      private:
            float re;
            float im;
      public:
            Complex(float r, float i) : re(r), im(i){};
            void Print();
            friend int IsEqual(Complex* c1, Complex* c2);
};

int IsEqual(Complex* c1, Complex* c2)
{
            if (c1->re == c2->re && c1->im == c2->im)
            {
                  return 1;
            }
            else
            {
                  return 0;
            }
}

int main()
{
            Complex* c1 = new Complex(1.0,2.0);
            Complex* c2 = new Complex(1.0,2.0);
            IsEqual(c1, c2);
            delete c1;
            delete c2;
            return 0;
}
```

U nekim slučajevima potrebno je da klasa ima “povlašćene” korisnike koji mogu da pristupaju privatnim i zaštićenim (protected) članicama klase. Tada klasa proglašava neke funkcije i/ili klase *prijateljima*. Prijateljstvo je, dakle, relacija koja reguliše pravo pristupa

Prijateljske funkcije su funkcije koje nisu članice klase, nemaju pokazivač **this,** ali imaju dozvoljen pristup privatnim članovima klase.

Prijateljske funkcije mogu da budu: globalne funkcije ili članice drugih klasa.

Šta friend deklaracija radi

To je deklaracija unutar klase kojom **autor klase** imenuje funkciju kao prijateljsku. Deklaracija može stajati u bilo kojoj sekciji — privatnoj ili javnoj (mjesto deklaracije nema značaja).

Ne narušava se princip enkapsulacije: projektant klase sam određuje koje funkcije će biti prijateljske. Izbjegava se “publikovanje” članova svim ostalim klasama.

Ključna riječ friend

```
class B;

class A
{
     public:
          int Func1( B& b );
          int Func2( B& b );
};

class B
{
     private:
           int _b;
           friend int A::Func1( B& );
};

int A::Func1( B& b ) { return b._b; } // OK

int A::Func2( B& b ) { return b._b; } // Greska jer Func2 nije prijatelj klase B
```

Primjer prijateljske funkcije koja je **metoda druge klase**

Prijateljska klasa

- Ako klasa X deklariše klasu Y kao prijateljsku, onda su sve metode klase Y prijateljske funkcije klasi X. Tada bilo koja metoda klase Y može da pristupa svim članovima klase X (obratno ne važi).

- Ako su sve metode klase Y prijateljske funkcije klasi X, onda je i sama klasa Y prijateljska klasa (friend class) klasi X

- Princip prijateljske klase se koristi kada dvije klase tijesno sarađuju
  - Npr. može se obezbijediti da samo klasa Y kreira objekte klase X: to se postiže tako što se konstruktori klase X deklarišu kao privatni, a zatim se klasa Y deklariše kao prijateljska klasi X

Primjer upotrebe - prijateljska klasa i privatni konstruktor

Račun smije otvoriti samo banka.

```
class Racun {
private:
    std::string broj;
    double stanje;

    Racun(std::string broj, double pocetno)   // privatan konstruktor
        : broj(broj), stanje(pocetno) {}

public:
    void prikazi() const;
    friend class Banka;
};

class Banka {
public:
    static Racun otvori(std::string broj)
    {
        return Racun(broj, 0.0);   // Banka je prijatelj klase Racun
    }
};
```

Kada pokušamo kreiranje objekta klase Racun iz funkcije main dobijamo grešku jer je konstruktor privatan pa mu main nema pristup

Racun a("501-003", 100);

error: 'Racun::Racun(std::string, double)' is private within this context

Jedino u metodama klase Banka se može kreirati objekat klase Račun.

U main-u bismo pisali ovako:

Racun a = Banka::otvori("501-003");

Pravila o relaciji prijateljstva

Nije simetrična

Ako je **Banka** prijatelj klase **Racun**, to ne znači da je **Racun** prijatelj klase **Banka**. Prijateljstvo se definiše u jednom smjeru.

Nije tranzitivna

Prijatelj mog prijatelja nije nužno moj prijatelj. Svaka klasa imenuje svoje prijatelje sama.

Ne nasljedjuje se

Klasa izvedena iz prijatelja nije prijatelj.

Prijatelj osnovne klase ne postaje automatski prijatelj izvedene.

Prijateljstvo treba koristiti rijetko

Prijateljstvo je izuzetak koji autor klase pravi namjerno, najčešće radi preklapanja operatora. Ako se spisak prijatelja produžava, to je znak da interfejs klase nešto ne nudi a trebalo bi.

Konstruktor sa jednim argumentom omogućava implicitnu konverziju

Ako klasa X ima konstruktor koji se može pozvati sa jednim stvarnim argumentom tipa T,
ovim konstruktorom se definiše korisnička konverzija iz tipa T u tip X

Tip T može biti ugrađeni ili korisnički definisan tip

Implicitno se koristi tamo gdje je potrebna, pri inicijalizaciji objekta tipa X objektom tipa T

```
class Kamata {
    double stopa;
public:
    Kamata(double stopa) : stopa(stopa) {}
    double getStopa() const { return stopa; }
};

void primijeni(const Kamata& k);

Kamata k = 3.0;      // double 3.0 postaje Kamata
primijeni(2.5);      // double 2.5 postaje Kamata
```

Ključna riječ explicit

```
class Kamata {
    double stopa;
public:
    explicit Kamata(double stopa) : stopa(stopa) {}
};

Kamata k = 3.0;         // zbog explicit ovo je greska
primijeni(2.5);         // zbog explicit ovo je greska

Kamata k(3.0);              // ovako se konverzija traži izricito
primijeni(Kamata(2.5));     // ovako se konverzija traži izricito
```

```
error: conversion from 'double' to
non-scalar type 'Kamata' requested

error: invalid initialization of reference
of type 'const Kamata&' from expression
of type 'double'
```

Ako želimo konstruktor sa jednim argumentom a ne želimo da omogućimo implicitnu konverziju, onda koristimo ključnu riječ explicit.

Oznaka **explicit** zabranjuje implicitnu konverziju.

Preporuka

Svaki konstruktor koji se može pozvati jednim argumentom označiti sa **explicit**, osim kada je konverzija zaista željena i očekivana.

Konstanta koja pripada klasi

```
class Racun {
public:
    static constexpr int maxTransakcija = 5;   //konstanta poznata pri prevodjenju

private:
    //može se koristiti npr. za dimenziju niza na steku
    double zadnje[maxTransakcija];
    int broj = 0;

public:
    void dodaj(double iznos)
    {
        if (broj < maxTransakcija)
            zadnje[broj++] = iznos;
    }

    void prikazi() const
    {
        std::cout << "zadnjih " << broj << " od " << maxTransakcija << ": ";
        for (int i = 0; i < broj; i++)
            std::cout << zadnje[i] << " ";
        std::cout << "\n";
    }
};
```

Max broj transakcija nije svojstvo pojedinog računa nego pravilo banke. Zato ga uvodimo kao statički clan a pošto je konstanta dodajemo constexpr

static — jedan primjerak za cijelu klasu. **constexpr** — vrijednost je poznata pri prevođenju.

Za **statički constexpr član** ne piše se definicija u .cpp datoteci, za razliku od običnog statičkog atributa kakav je brojRacuna

Statička metoda koja pravi objekat

Broj računa ne bi trebalo da zadaje korisnik klase. Klasa ga može dodijeliti sama, jer jedino ona zna koliko ih je do sada otvorila (preko statičkog brojača).

```
class Racun {
    std::string broj;
    std::string vlasnik;
    double stanje;
    static int brojRacuna;

public:
    static Racun otvori(std::string ime)
    {
        std::string novi = "501-"
            + std::to_string(1001 + brojRacuna);
        return Racun(novi, ime, 0.0);
    }
};
```

U main.cpp:

```
...
Racun a = Racun::otvori("Marko Markovic");
Racun b = Racun::otvori("Ana Anic");
```

Objekat joj nije potreban da bi radila — naprotiv, ona ga pravi. Ovakve metode zovu se **fabričke**, i to je druga česta svrha statičke metode, pored brojača.